NDTwin progress · 2026-09-30

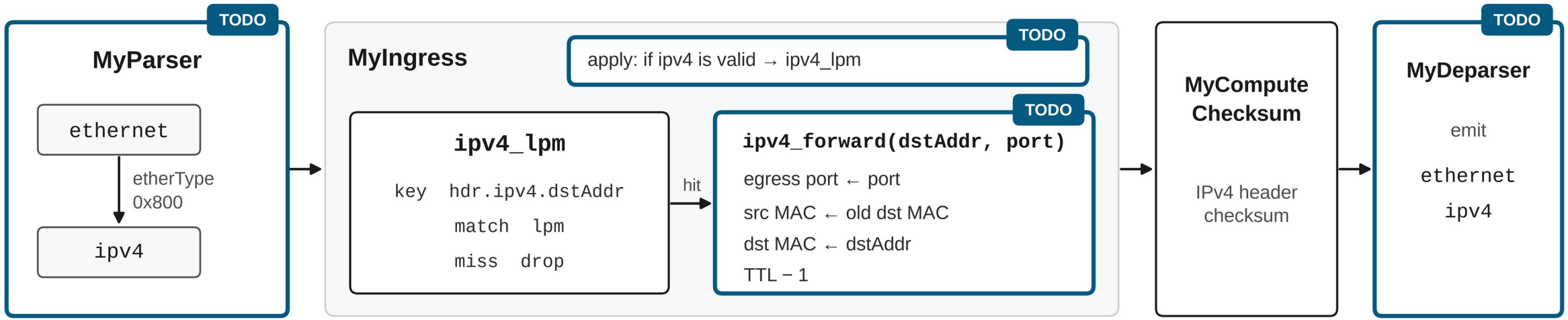
Adam

The 13 P4 tutorial exercises

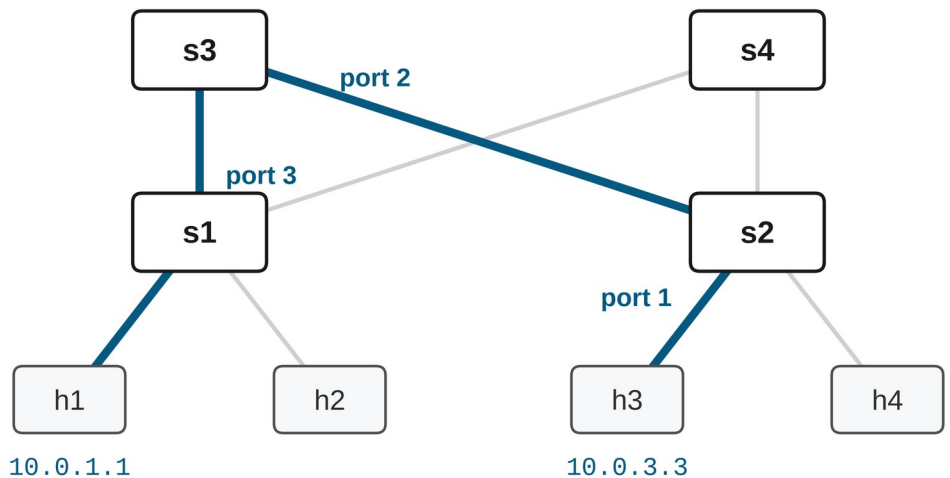
exercise	the student writes	the packet carries	its tables are filled by
basic	lpm lookup, MAC rewrite, TTL − 1	IPv4	sN-runtime.json
basic_tunnel	a tunnel header and its own exact table	myTunnel 0x1212 + IPv4	sN-runtime.json
calc	a calculator that answers the sender	calc header 0x1234	none: constant entries in the .p4
ecn	ECN marking when the queue is deep	IPv4, ECN bits	sN-runtime.json
firewall	a stateful firewall (bloom filter) on s1	IPv4 + TCP	sN-runtime.json; s1 has its own program
flowcache	packet-in on a miss; idle timeout	IPv4; packet-in/out header	its own controller
link_monitor	probes that collect link utilization	probe 0x812 + header stacks	sN-runtime.json
load_balance	a 5-tuple hash picks one of two hosts	IPv4 + TCP	sN-runtime.json
mri	per hop: switch ID + queue depth	IPv4 option + header stack	sN-runtime.json
multicast	MAC lookup; a miss floods a group	Ethernet (ARP, IPv4)	sN-runtime.json + 1 multicast group
p4runtime	a controller that writes tunnel rules	myTunnel 0x1212 + IPv4	its own controller
qos	DiffServ marking by protocol	IPv4, DSCP bits	sN-runtime.json
source_routing	pop one output port per hop	route stack 0x1234	none: the path is in the packet

basic: one packet through the program

READ · p4lang/tutorials c80d83e



pod-topo/topology.json · h1 → 10.0.3.3



pod-topo/s1-runtime.json · installed at start

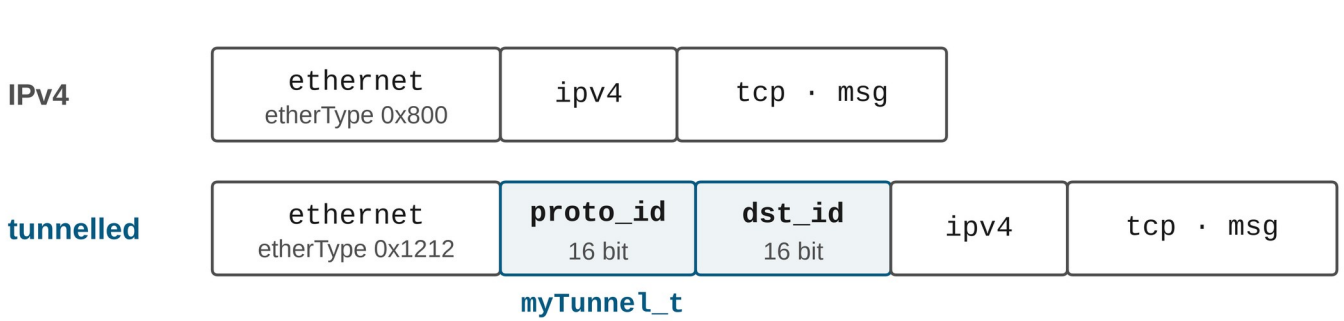
hdr.ipv4.dstAddr	action	dstAddr	port
10.0.1.1/32	ipv4_forward	08:00:00:00:01:11	1
10.0.2.2/32	ipv4_forward	08:00:00:00:02:22	2
10.0.3.3/32	ipv4_forward	08:00:00:00:03:00	3
10.0.4.4/32	ipv4_forward	08:00:00:00:04:00	4
default	drop		

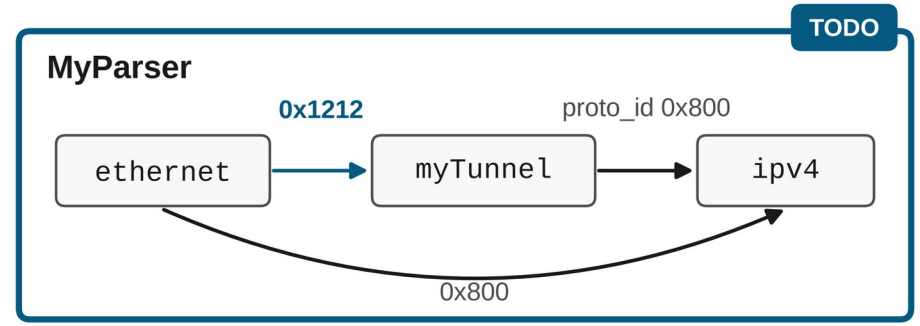
TODO you write it: the skeleton leaves a TODO

skeleton: drops every packet

basic_tunnel: its own header, its own table

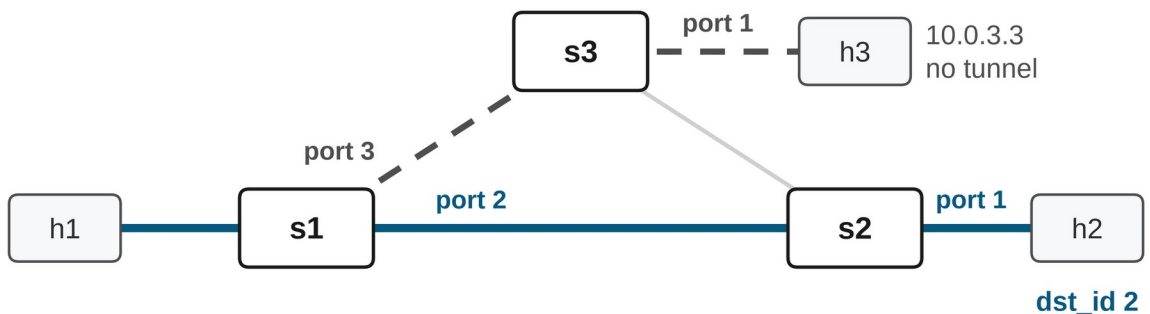
READ · p4lang/tutorials c80d83e







topology.json · h1 sends to 10.0.3.3



s1-runtime.json · installed at start

myTunnel_exact: dst_id	action	port
1	myTunnel_forward	1
2	myTunnel_forward	2
3	myTunnel_forward	3

+ 3 ipv4_lpm entries

TODO you write it: the skeleton leaves a TODO

What the 09-16 NDTwin lacked for them

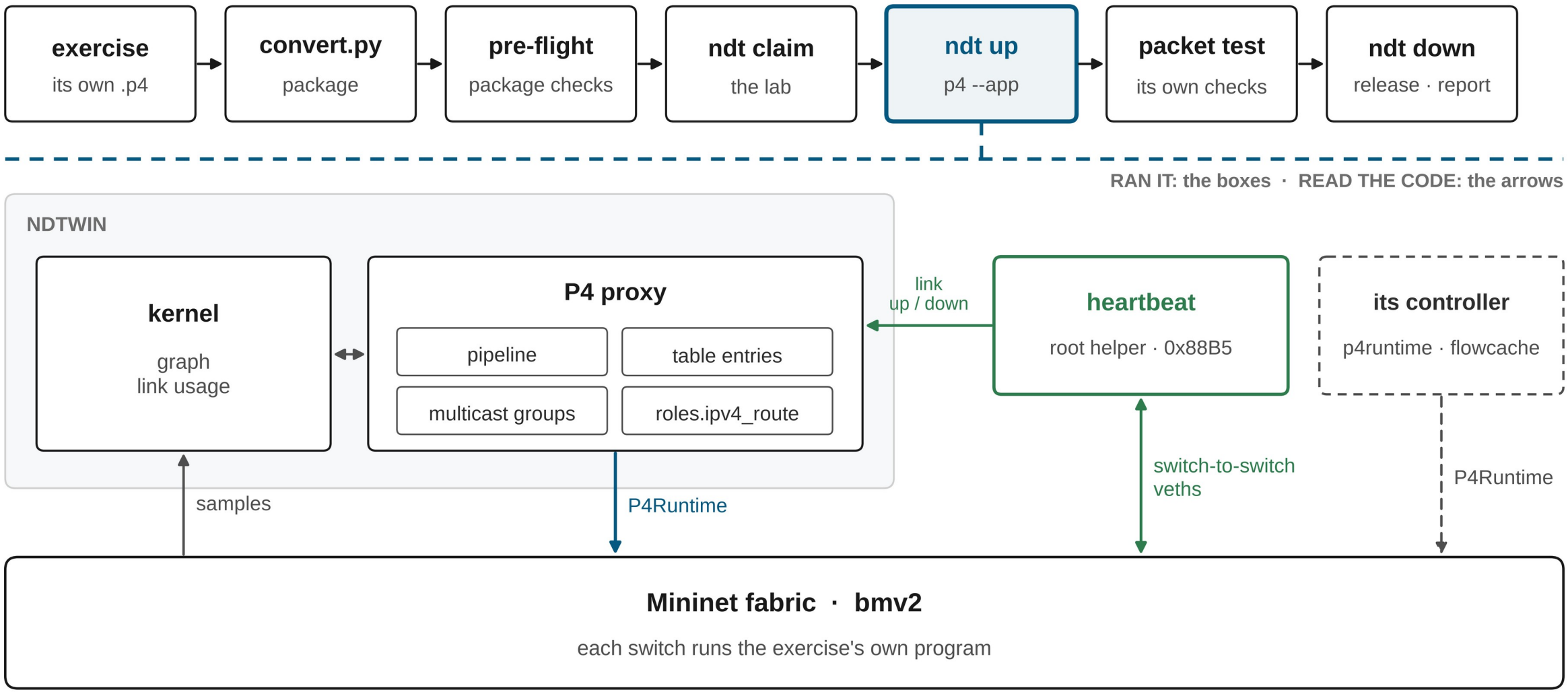
need	NDTwin on 09-16	what that blocked
own program per switch	every bmv2 ran ndtwin_switch.p4	all 13 (firewall: two programs)
entries by its own names	fixed names: ipv4_lpm, flow_5tuple	9 exercises' start-up entries
multicast groups	no way to write one	multicast
its own topology	10 switches fixed; hosts by formula; no shaping	other shapes, 1 switch; ecn/mri queues
link up/down detection	LLDP via packet-out/in: needs ndtwin_switch.p4	link state on any other program
NDTwin routes in its table	/32 routes into its own ipv4_lpm	NDTwin routing on its program
its own controller	one fixed election id for every client	p4runtime, flowcache
link usage of its traffic	samples cloned by ndtwin_switch.p4	every link rate 0, no error
what a switch cannot do	no capability field	writes that fail after "queued"

How each gap was bridged

need	decision	mechanism	ran live	not yet
own program per switch	the exercise comes as a package (09-17)	convert.py → package; ndt up p4 --app	13/13 exercises (T06)	—
entries by its own names	names checked before anything starts	pre-flight; proxy writes by p4info names	9 exercises, 0 failed	not done: ternary, range
multicast groups	the package declares them	proxy writes them over P4Runtime	multicast: 1 group	—
its own topology	the package carries it	ndtwin/topology.json; shaping if declared	pod, triangle, 5h/3s, 1 switch	—
link up/down detection	no .p4 change (09-24)	veth heartbeat from a root helper	6/6 cuts detected (08)	not done: p4runtime, flowcache
NDTwin routes in its table	roles: NDTwin owns one table (09-24)	proxy binds its route writes to it	basic (07, twice)	offline: 6 same-shape exercises
its own controller	the proxy stays out	an adapter points it at the fabric	p4runtime, flowcache	—
link usage of its traffic	sample the links, not the program	tc sampling on switch veths → sFlow	10/13 (plain IPv4 iperf)	not run: 3 (drop plain iperf)
what a switch cannot do	capabilities, read once (09-24)	switch_state → get_graph_data → GUI	proxy (07, twice)	kernel: unit tests · GUI: local draft

How a foreign P4 program runs on NDTwin

RAN IT · every step appears in T06's arm reports



What NDTwin covered: 09-16 vs 09-27

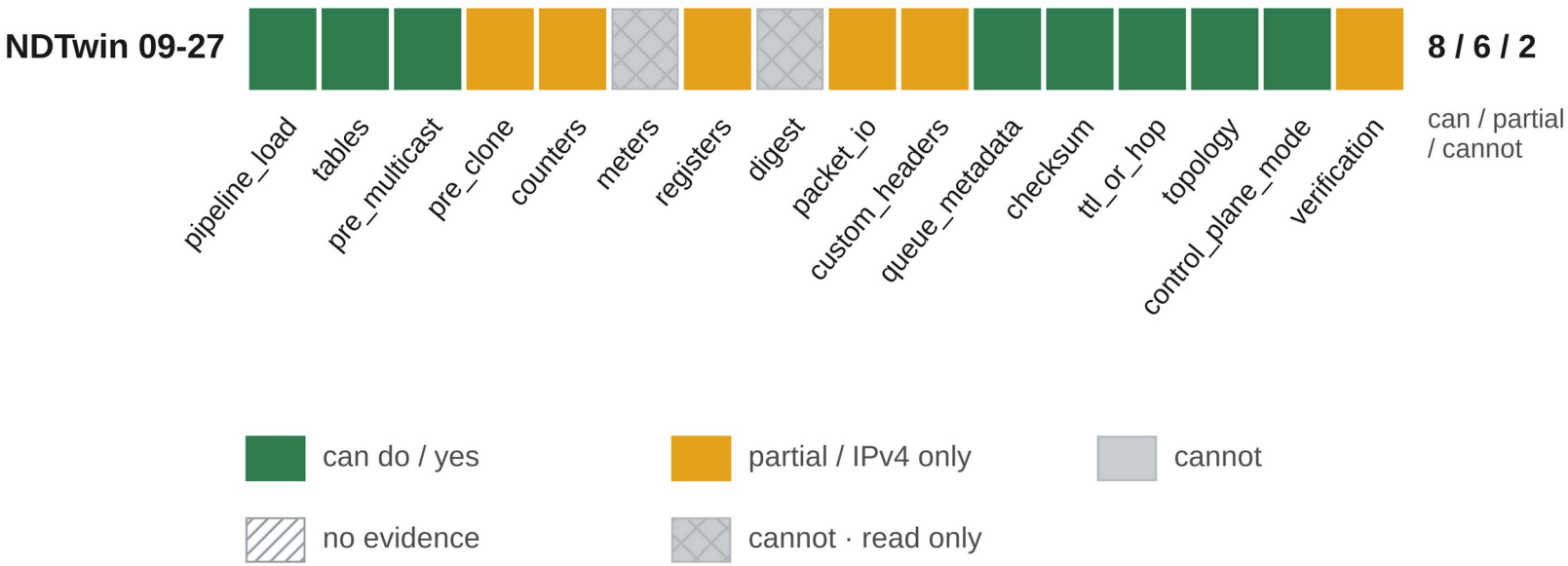
as of GAP-2b with Adam's 2026-09-28 ruling

BACKGROUND · READ THE CODE · NDTwin's own P4

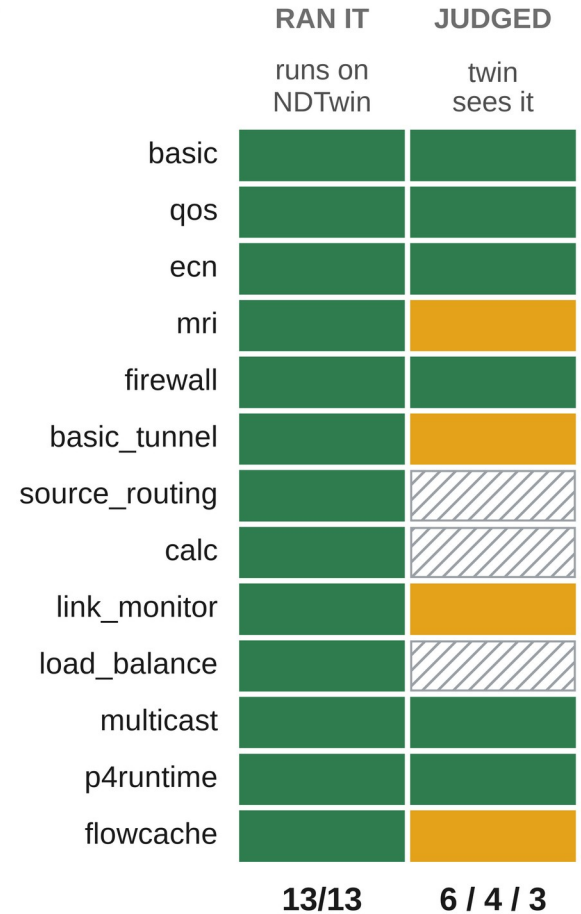


as shown 09-16; read 09-08, trunk 1a284f75

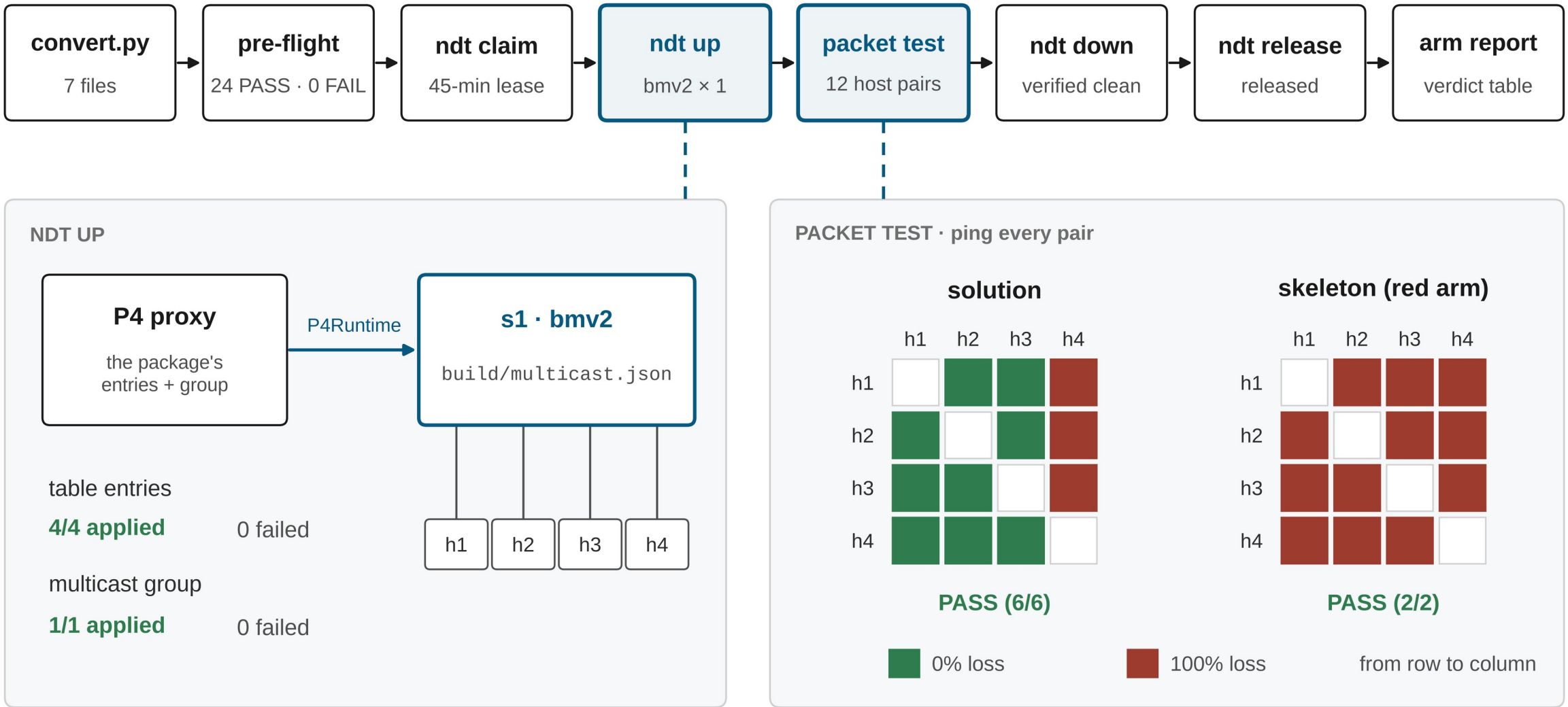
RAN · JUDGED · 09-27 · each exercise's own P4 on the NDTwin fabric



T06 · 09-27



multicast, end to end on the NDTwin fabric



What pre-flight checks before ndt up

package & control plane

format	26
name	26
control_plane.mode	26
control_plane.grpc_base	26
control_plane.device_id	26
control_plane.election_id	26
bmv2.cpu_port	26
switches keys	26
switches name	26
referenced files	26

topology agrees

topo_from_json.switches	26
topo_from_json.hosts	26
topo_from_json.switch_links	26
topo_from_json.host_links	26
switches agree	26
links agree	26
hosts named h<last octet>	26
hosts agree	26

pipeline & p4info

switches pipeline	26
sN pipeline	26
p4info parses	26
telemetry.source	26

entries

entries		3
entries match p4info	22	1
entries p4info is the pipeline's		23
PRE entries	2	24

roles

roles.ipv4_route resolves	1
owned table has no package entries	1
owned table default action	1
roles suggestion	16

ports & compile

gRPC port block	26
p4c-bm2-ss	26

basic_tunnel · skeleton

FAIL entries match p4info

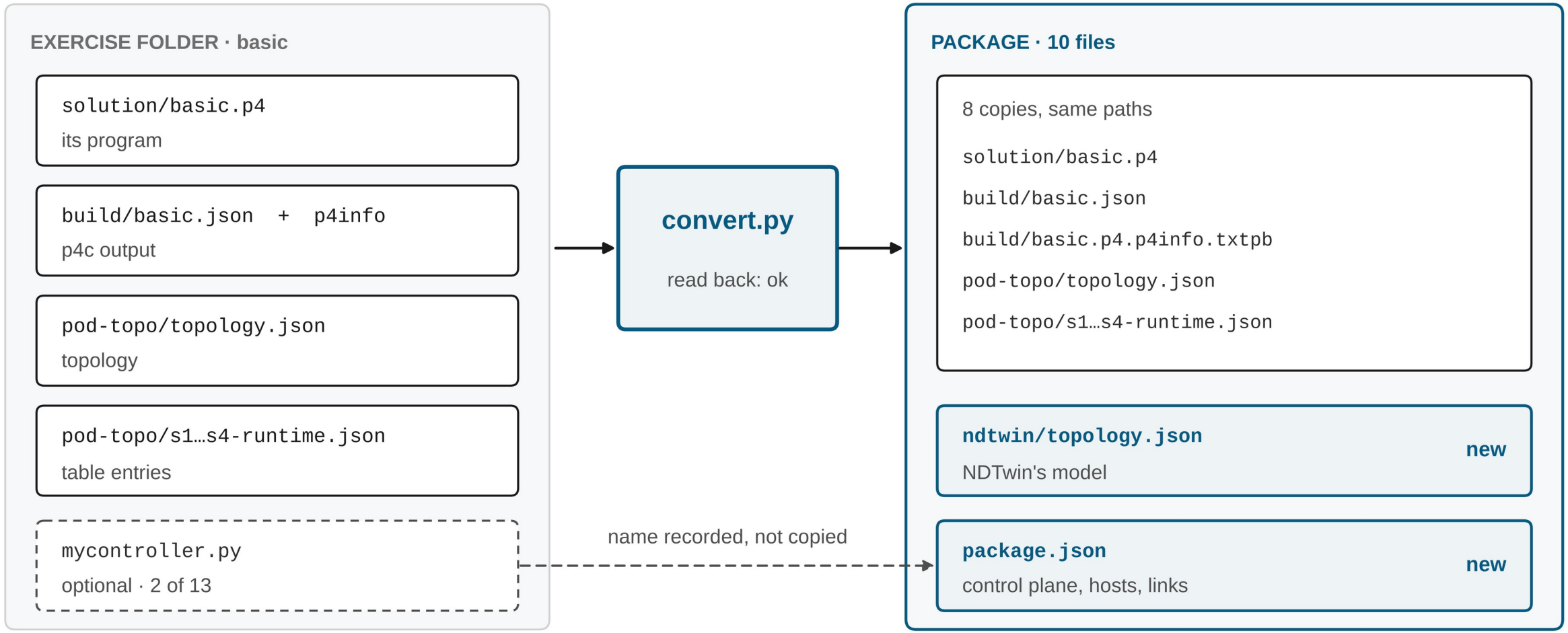
9 problems, first:

`MyIngress.myTunnel_exact`

not in its p4info

PASS INFO FAIL

What the exercise brings, what convert adds



Seven custom-header exercises forward

	RAN IT · 09-27 forwards (solution arm)	SAME RUNS' RAW · JUDGED twin saw its own frames
p4runtime 0x1212 tunnel	PASS (5/5)	yes · 0x1212 bytes
basic_tunnel 0x1212 tunnel	PASS (4/4)	IPv4 only
flowcache packet-in/out header	PASS (5/5)	IPv4 only
link_monitor 0x812 probe + stacks	PASS (4/4)	IPv4 only
mri IPv4 option + stack	PASS (5/5)	IPv4 only
calc 0x1234 + lookahead	PASS (3/3)	no evidence
source_routing 0x1234 + route stack	PASS (5/5)	no evidence

Four rulings on the P4 API shape (09-24)

ruling (09-24)	ran live	offline / read only	not done yet
① **owner: ndtwin → NDTwin owns that table**	pass path: 0 package entries, 16/16 routes written (07, twice)	refusing package entries: unit test + caught mutant	a live refusal; "rerouted table derivable from the declaration"
② **no .p4 changes; root veth heartbeat**	owned route tables: detect + reroute (08 H1–H5)	tutorials' own basic.p4 unmodified	external control planes: no heartbeat; worst watchdog phase
③ **GUI reads capabilities once, greys out**	proxy reports capabilities, 4/4 switches, both 07 runs	kernel get_graph_data carries them: unit tests + CI (merged 09-28); GUI draft on a local branch	kernel side on a live fabric; grey-out in a pushed GUI (cut 3, GUI side)
④ **first cut: 7 same-shape programs, one table**	basic only (07, twice)	6 others: convert + pre-flight PASS; 4 other shapes FAIL as intended	the 6 on a fabric; multi-table, tunnel, ternary; no 5-tuple role

roles.ipv4_route: from package to table

RAN IT · live 07, 09-26

package.json `roles.ipv4_route`

```
table   MyIngress.ipv4_lpm
match   hdr.ipv4.dstAddr
action  MyIngress.ipv4_forward
```

convert.py

16

package entries taken out

pre-flight

PASS roles resolve

PASS no package entries

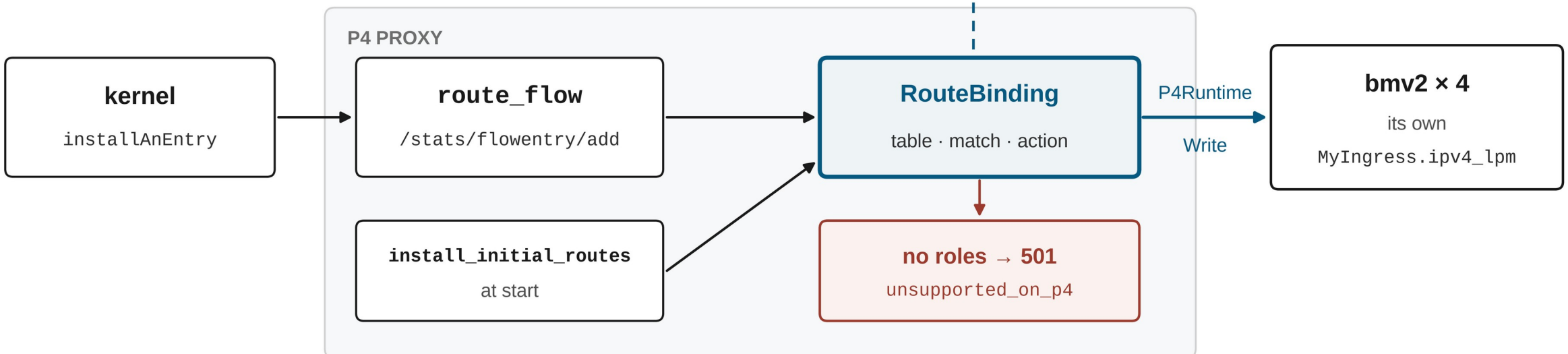
ndt up p4 --app

16 of 16

routes written by the proxy

`same resolve()`

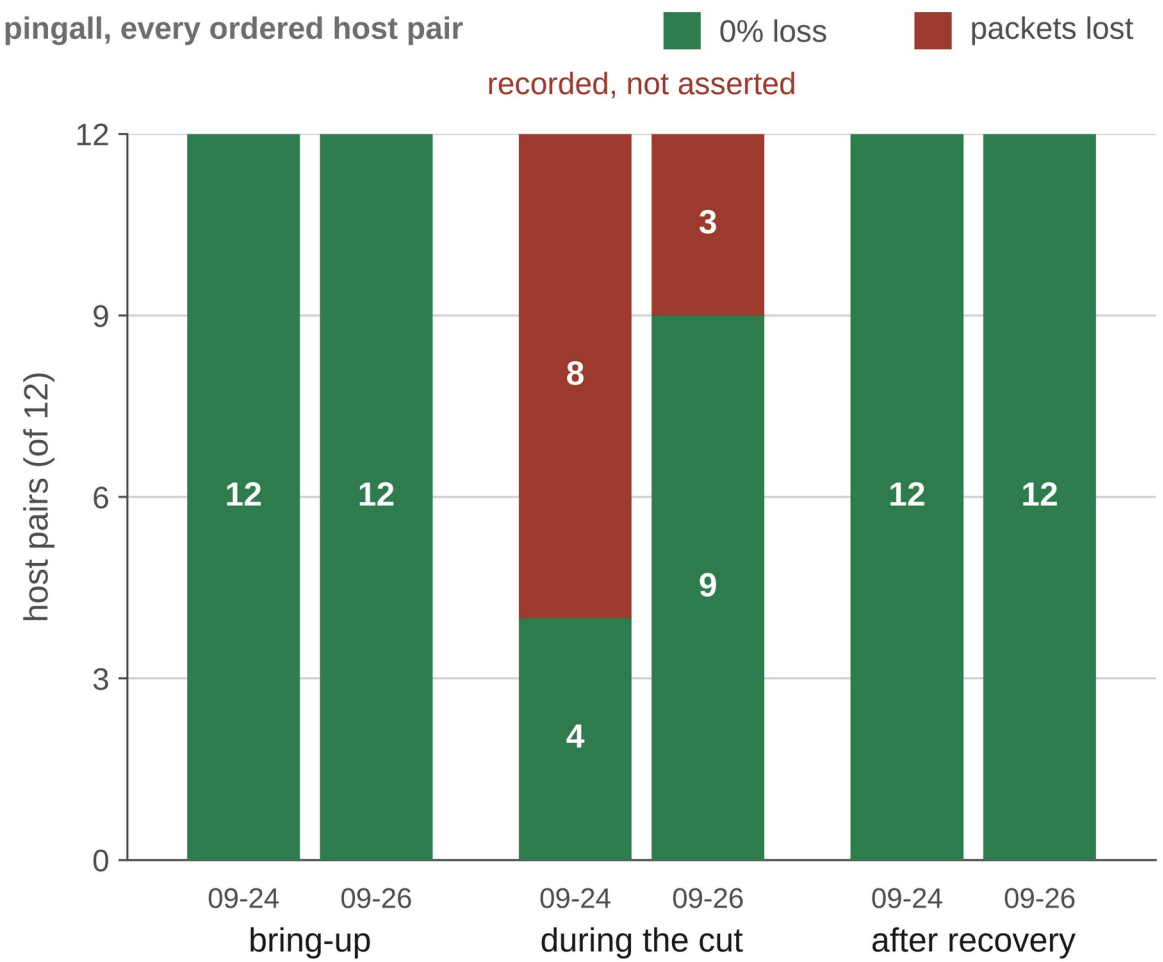
READ THE CODE · trunk 3d740be0



Live 07: roles on basic, run twice

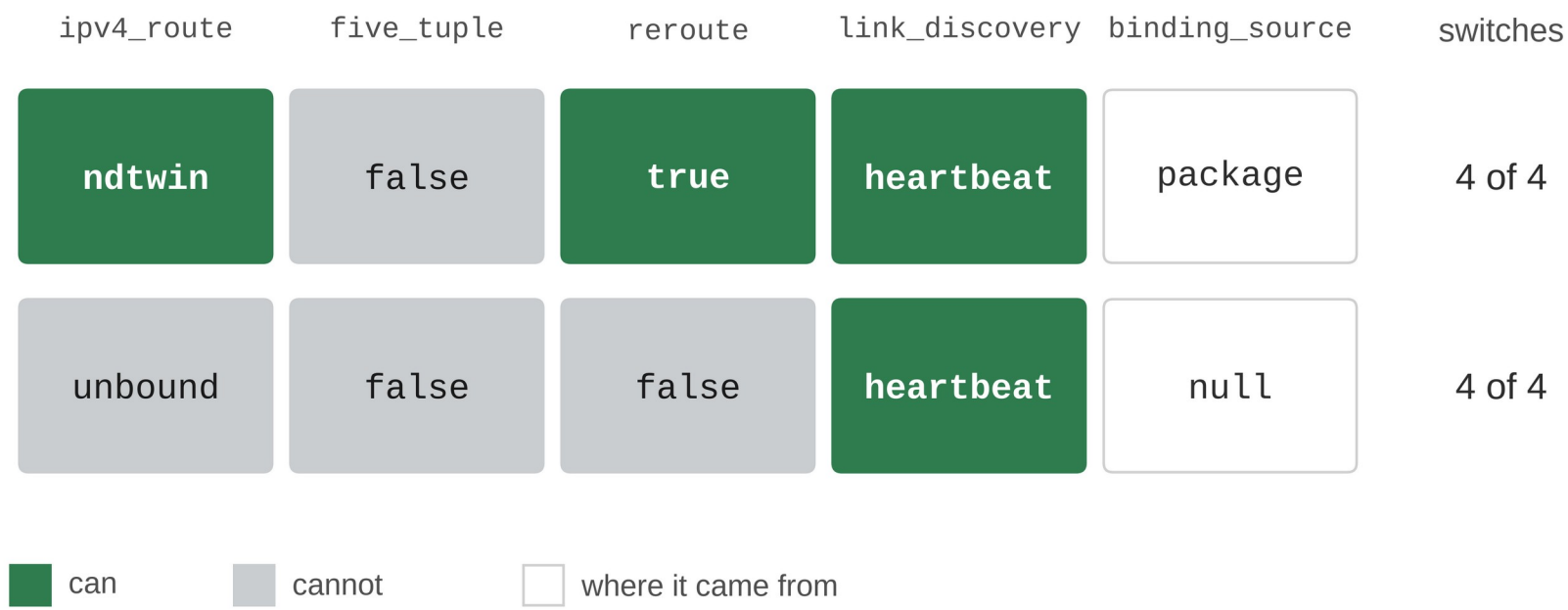
verdict per cell

		09-24 no heartbeat	09-26 heartbeat on
L1	links up in the graph	PASS 2 checks	PASS 3 checks
L2	kernel write, read back	PASS 3 checks	PASS 3 checks
L3	pingall at bring-up	PASS 1 check	PASS 1 check
L4	cut held, recovered	PASS 4 checks	PASS 4 checks
L5	no roles → 501	PASS 6 checks	PASS 6 checks
L6	capabilities	PASS 4 checks	PASS 4 checks

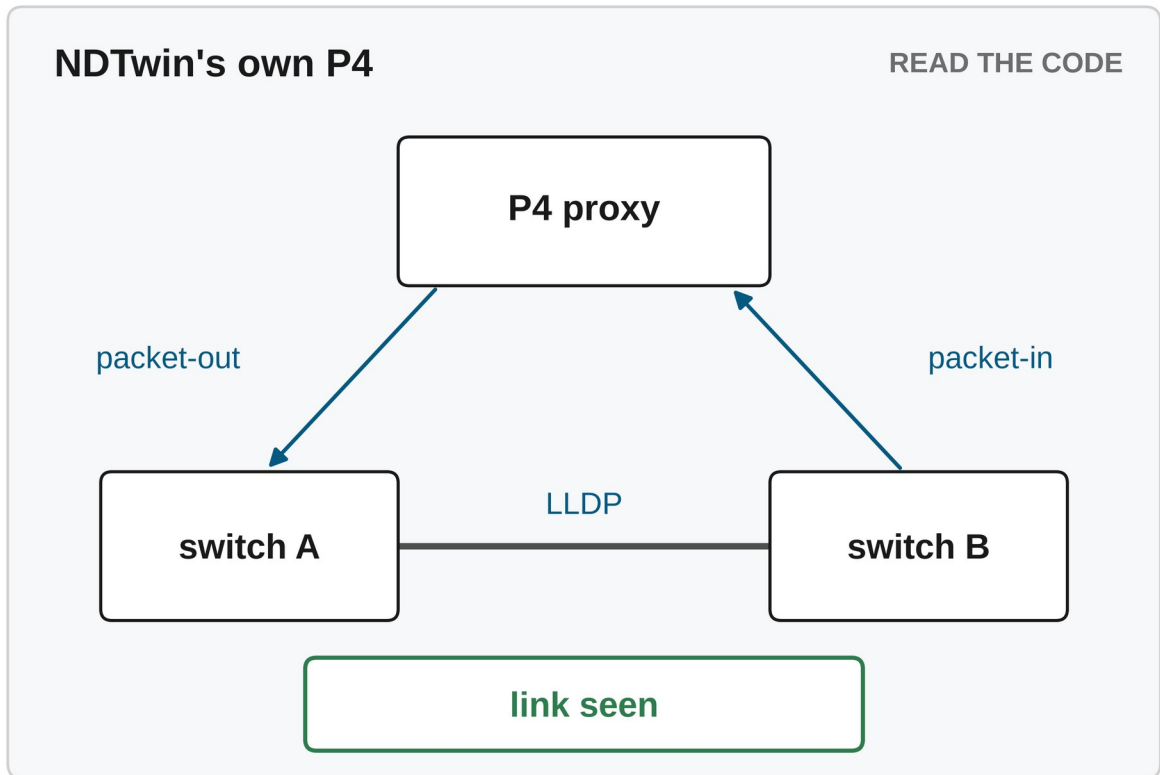


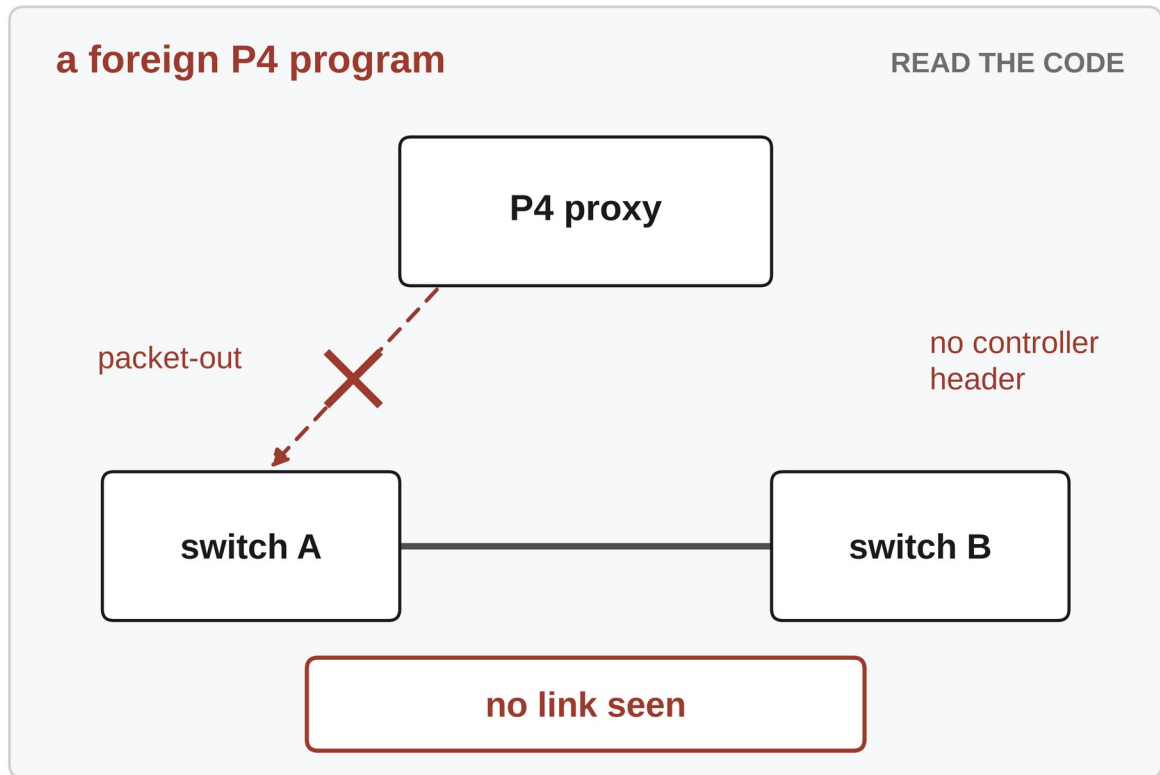
What each switch reports: capabilities

RAN IT · live 07, 09-26



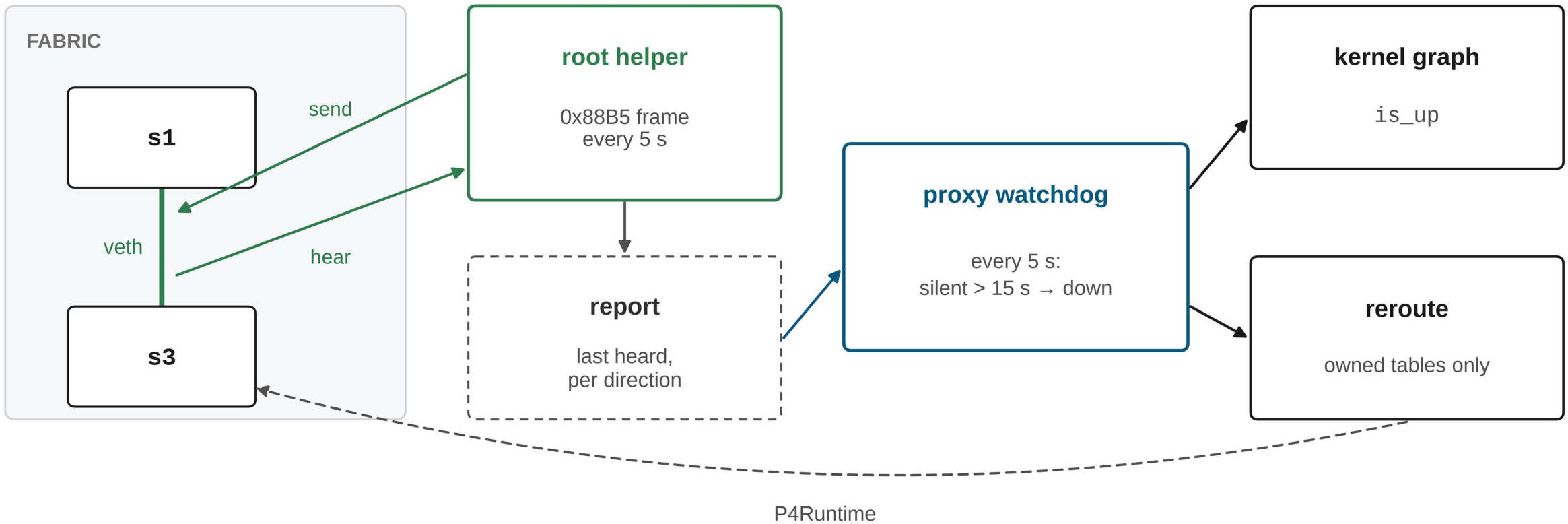
Why link discovery stops at a foreign P4



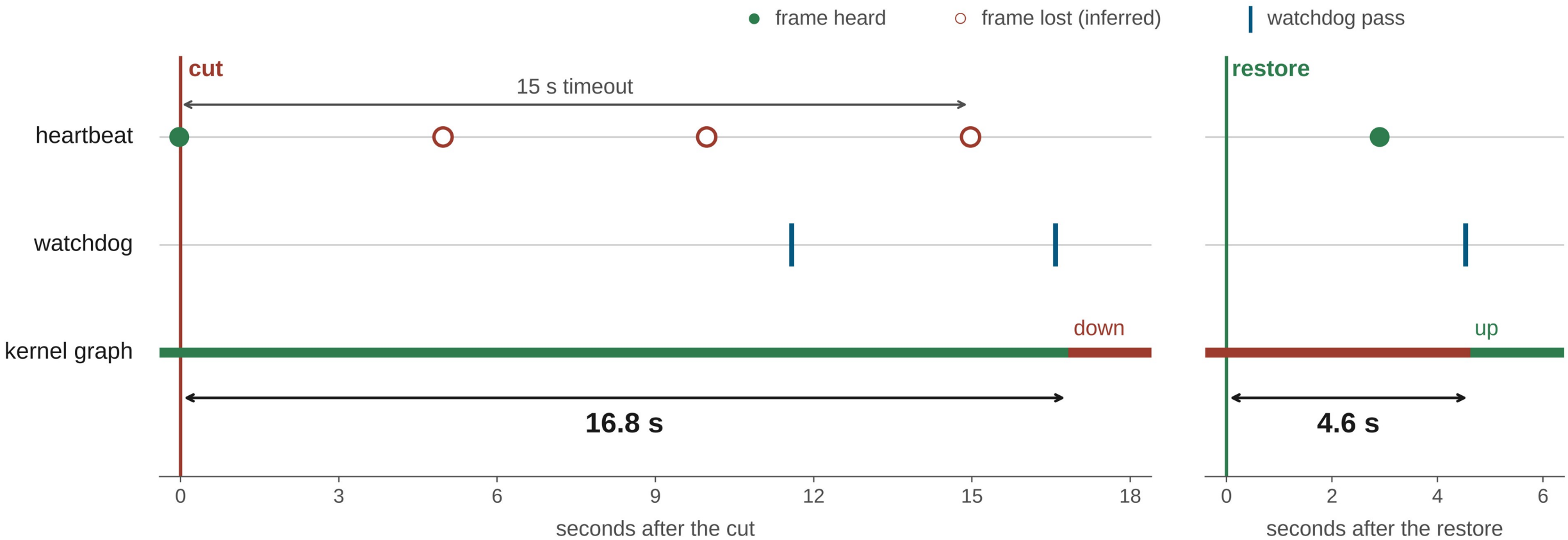


RAN IT · 09-24, before the heartbeat · skipped on a foreign fabric: `install_initial_routes` · `link_watchdog` · `lldp_discovery`

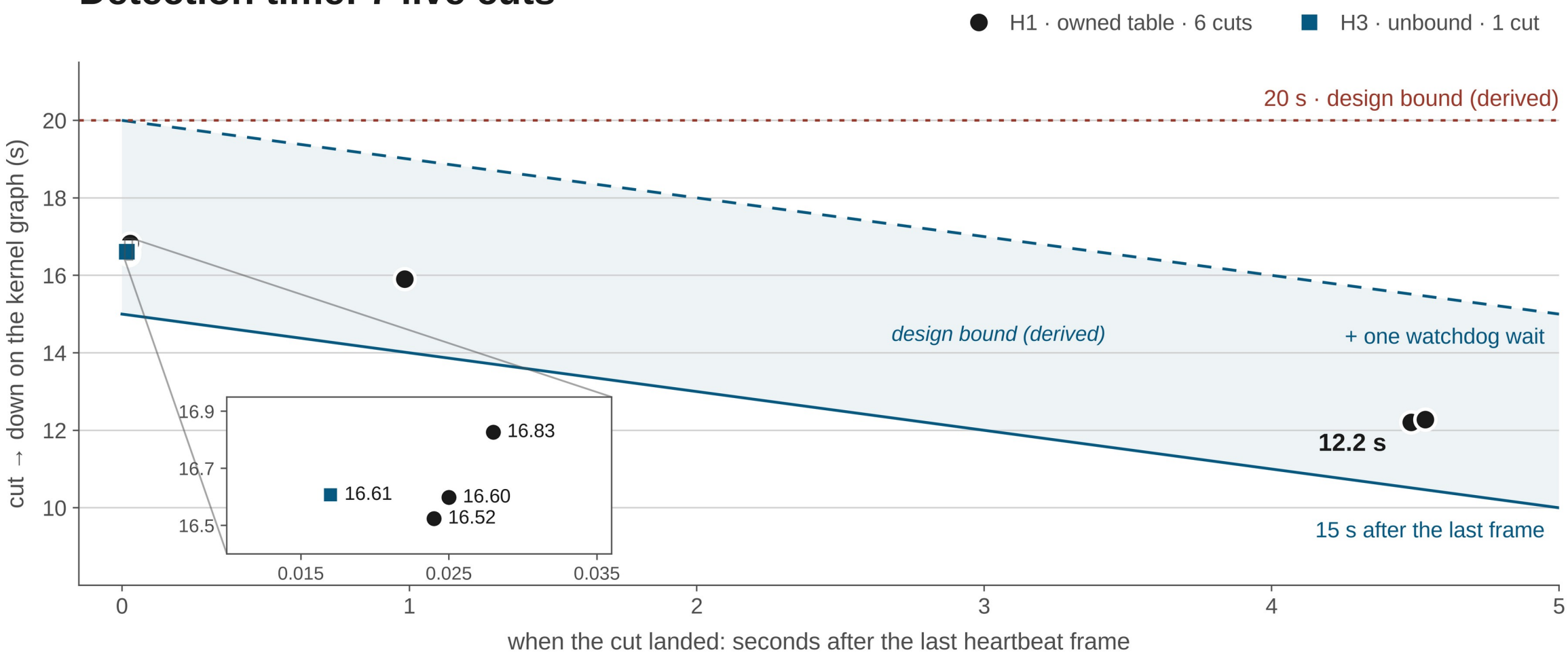
The veth heartbeat, from wire to kernel graph



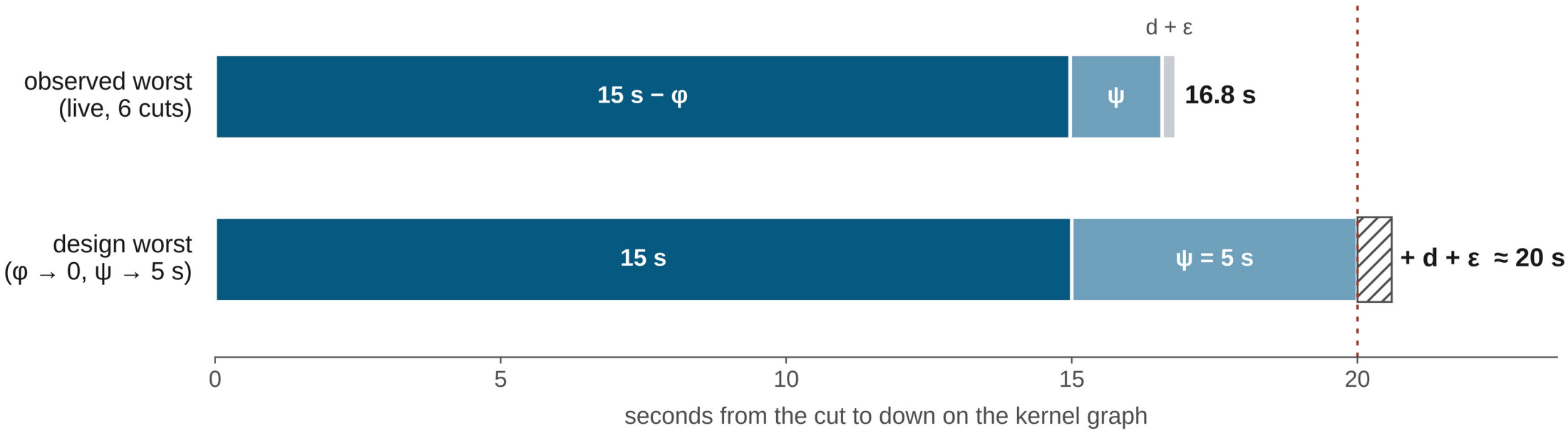
One cut, one restore — on the kernel graph



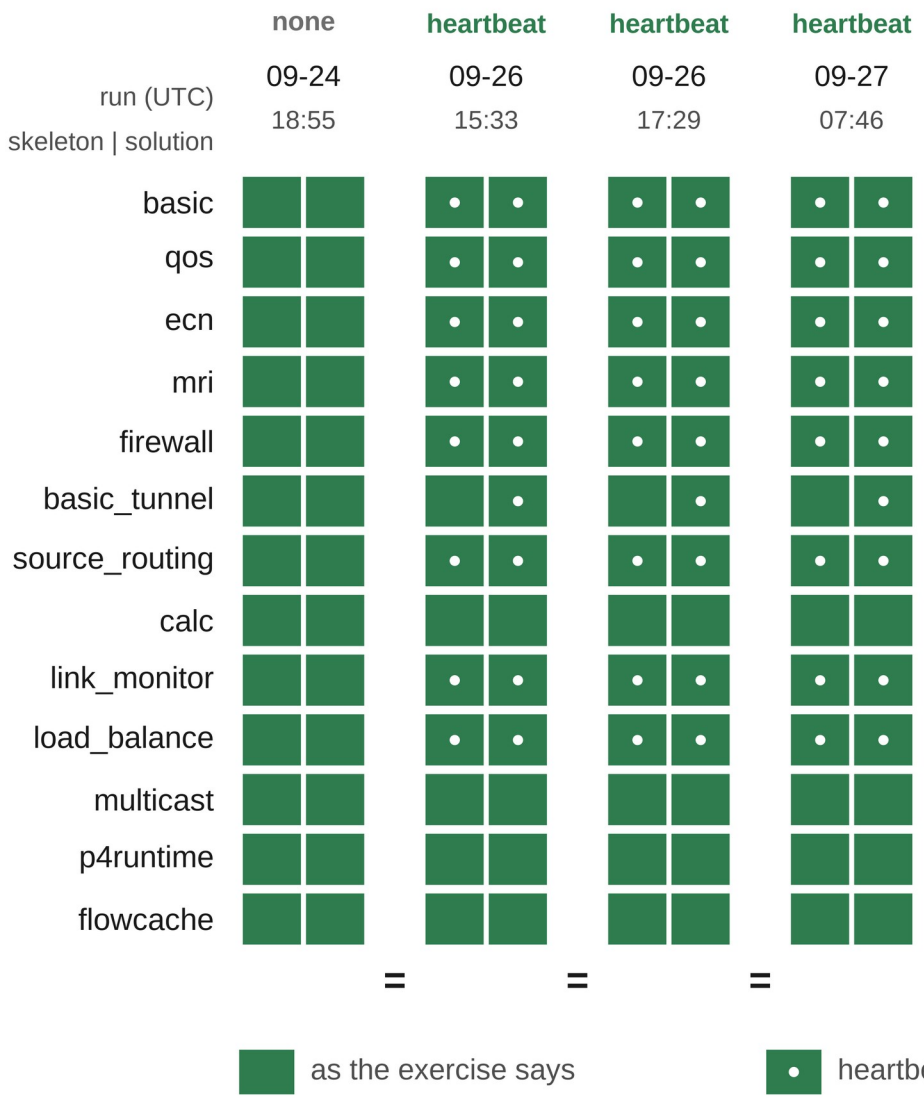
Detection time: 7 live cuts



The detection bound, piece by piece



Heartbeat on, forwarding unchanged



0

heartbeat frames
leaving a host port

17 heartbeat sessions · run 09-26 17:29Z

Changing rules at runtime, per data plane

RAN IT

	install	modify	delete
foreign P4 + roles **09-26**	**yes** kernel API	not run	**yes** kernel API
foreign P4, no roles **09-26**	**refused** 501	not run	not run

BEFORE 09-16 · BACKGROUND

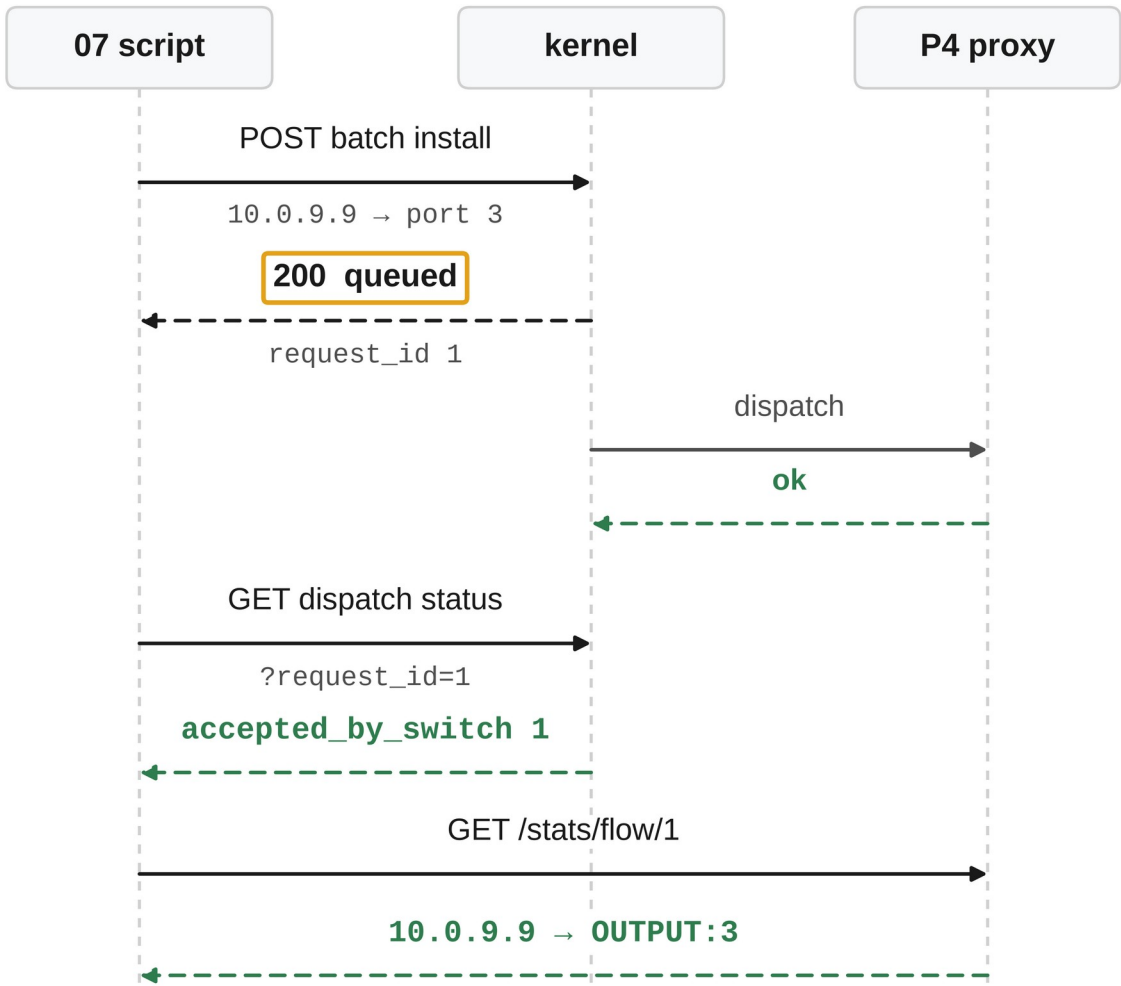
OVS **09-03**	**yes** kernel API	**yes** kernel API	**yes** kernel API
NDTwin's own P4 **08-24**	**yes** proxy only	not run	**yes** proxy only

READ THE CODE · trunk 3d740be0

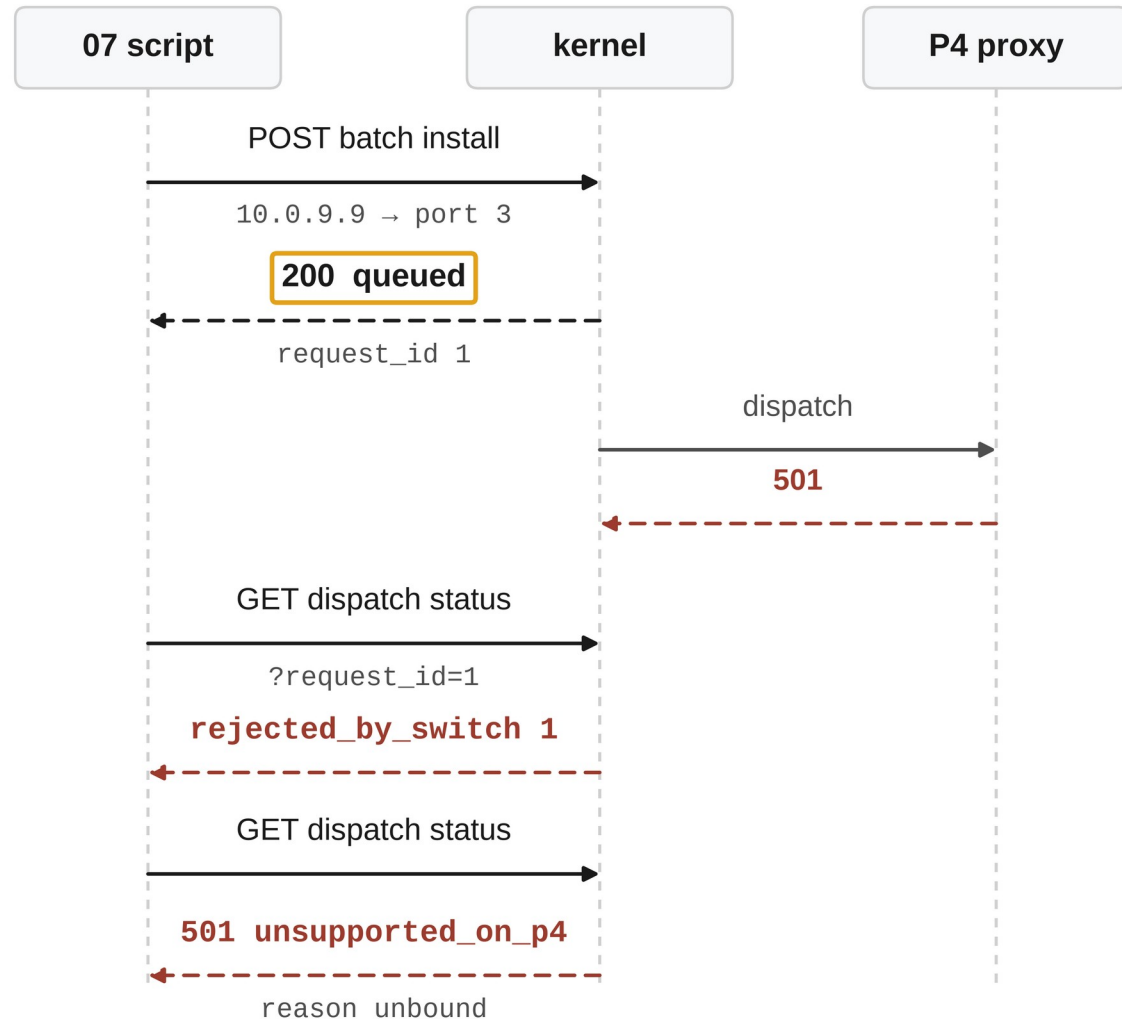
matches	actions
dst IP	the declared action
exact · lpm via /p4/table_entry	any in its p4info not journaled
OpenFlow match, passed on	passed on to Ryu
dst IP · 5-tuple	OUTPUT only

"queued" is not "installed": read it back

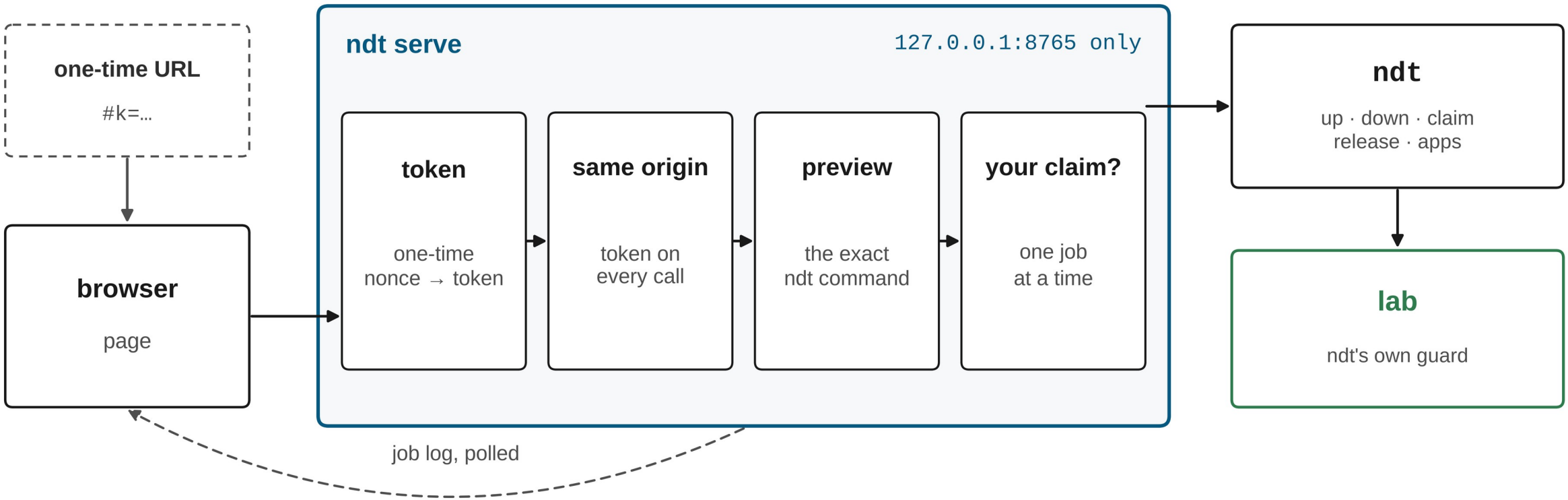
basic + roles



basic, no roles



ndt serve: one local door to the lab



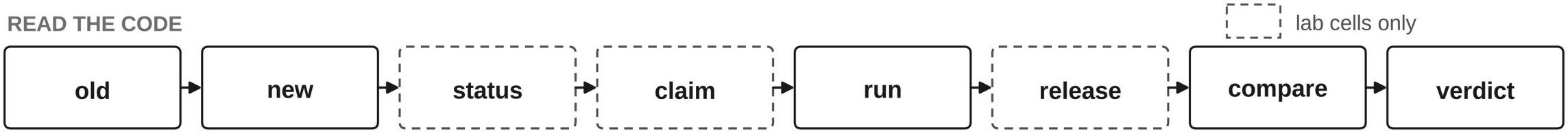
ndt serve: threats and defenses

threat	READ THE CODE	RAN IT · live 09-24 (UTC)
another machine	bind 127.0.0.1 only	**127.0.0.1:8765** 1 listener
a page rebinding DNS	Host header must be ours	**403** "host"
a page forging a call	token on every call own Origin · no CORS	**403** "token"
a leaked page link	one-time key · 600 s · one use	not run live
a surprise command	preview: the exact ndt argv	not run live
a second job mid-run	one job at a time	**409** "busy"
someone else's claim: up/down	ndt's own guard: rc 5 P4 and OVS	**P4 rc 5** refused / **OVS** not run live
someone else's claim: apps, cells	server: claim must be yours	not run live

held not run live

Live cells: two guided walks, 09-24

READ THE CODE



RAN IT · dates UTC

before 09-16 · background

`help_drops_deleted_claims`

no lab

old	09-11	FAIL	
new	09-11	PASS	
run	09-24	PASS	

red → green **4**

`up_target_names_a_readable_model`

lab · OVS 4 hosts

old	09-10	FAIL	
new	09-11	PASS	
run	09-24	PASS	

red → green **4 (+4 fixture gap)**

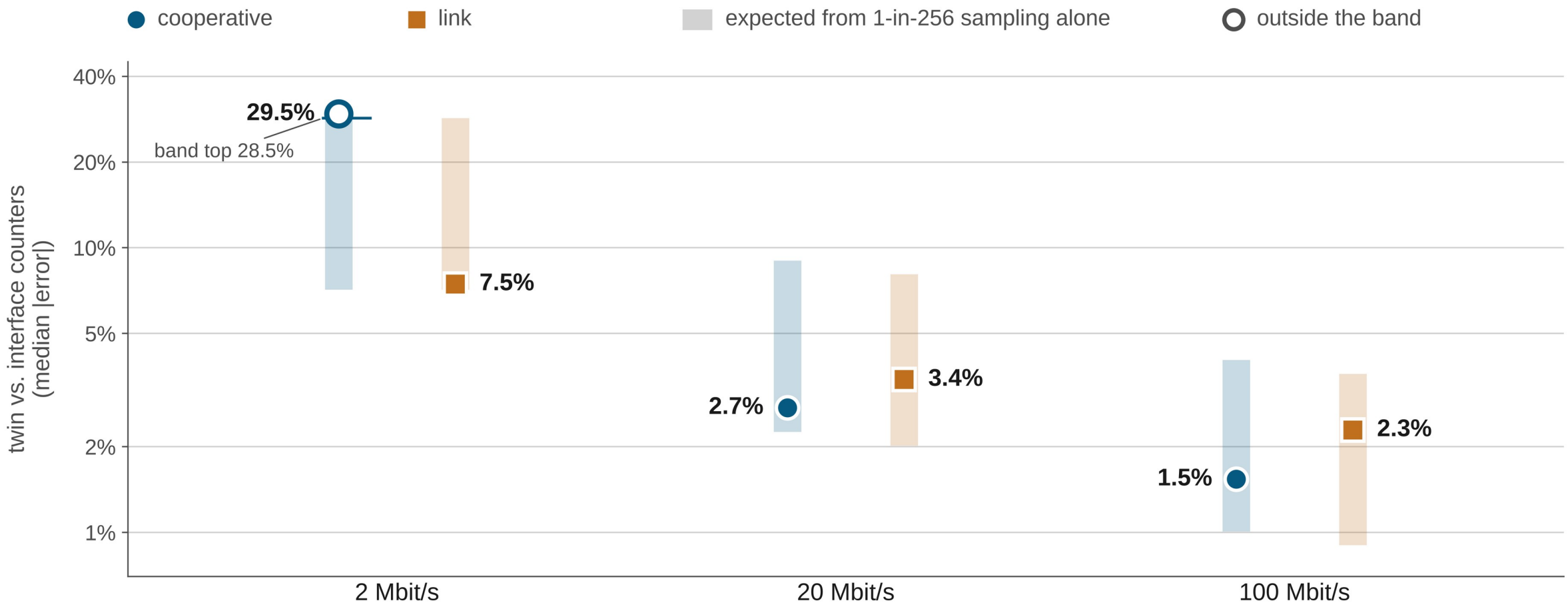
ok FAIL FAIL: fixture gap

one square = one assertion

Where the ndt serve page goes

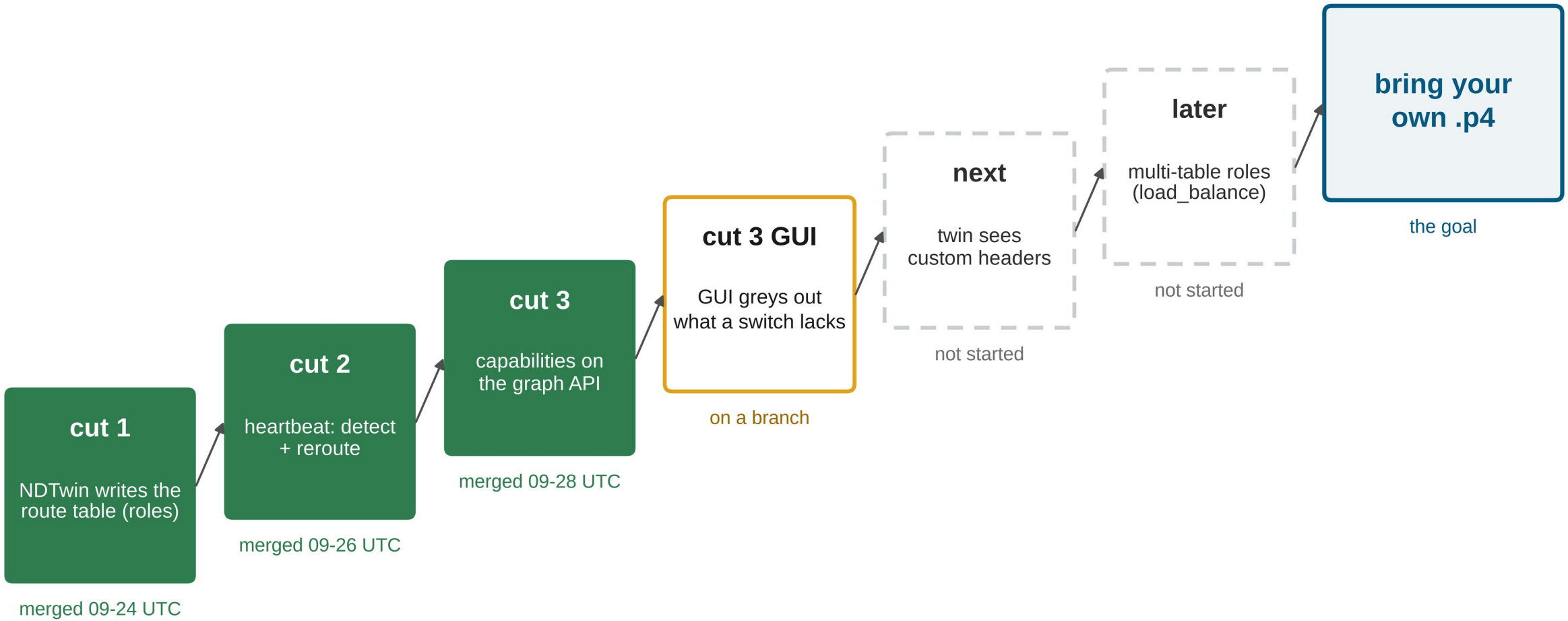
stage	stack	served by	refresh	state (UTC)
v1	static HTML + JS + CSS, no build step	ndt serve, 127.0.0.1 only	manual; job log re-read every 2 s	merged f186ce98, 09-27 12:35
v2	React · Vite · Tailwind · TypeScript · i18next	ndt serve, 127.0.0.1 only	every 10 s; paused while measuring	ruled 09-27; on a branch, no page yet
one app with Web-GUI	Web-GUI's	undecided	—	long-term; not scheduled

Sampling error of two telemetry paths



The road to “bring your own .p4”

as of trunk 3368412d · 2026-09-28 07:45 UTC



Open questions

question	why we ask	if yes	if no
next: custom headers, or multi-table roles?	one person, two roadmap items	observation side table → flow identity	load_balance template first
is "about 20 s" link detection enough?	same class as NDTwin's own LLDP	keep one shared watchdog	event-driven pass, ≈15.5 s design worst
re-measure the pps ceiling?	not resolvable on this machine	one night, a new preregistration	report sampling error only
ndt serve + Web-GUI as one app: which guard?	Web-GUI listens on 0.0.0.0:3000	loopback-only Web-GUI	login or an auth proxy
"bring your own .p4": what counts as done?	defines the next cuts	forwarding + link view	+ per-flow view of its own headers

What we do not claim

we do not claim	because	evidence
NDTwin operates any foreign P4	roles bind one dst-IP table; no link detection under an external controller	route_binding.py:159-161; h14 log :168
the twin observes custom headers	own-frame evidence only for 0x1212	T7; GAP-2b §3
telemetry lowers the pps ceiling	no resolution at 20 kpps	cooperative_f64_a.log :8, :15
detection ≤ 20 s, guaranteed	one watchdog phase sampled	30_cycles.tsv: phase 1.49–1.76 s only; design bound ≈ 20 s
flowcache G1 PASS = delivered	flow reached s1→s3 only; h3 got nothing	netdev s3-eth1 +273 B; iperf_server.txt; iperf_client.txt :10
the serve GUI ran on the lab	stub server only	ndt-serve-gui REPORT :10
users already have this work	the manual's clone target stops at 09-03	FIXED-SINCE-916 §8–§9